

A Clear Recursive Tree for Fibonacci

Understanding `fib(4)` and `fib(5)` step by step

Big idea: every non-base Fibonacci call splits into **two smaller calls**.

A call `fib(n)` first needs `fib(n-1)` and `fib(n-2)`, then it adds their returned values. The final outputs are `fib(4) = 3` and `fib(5) = 5`.

The Code

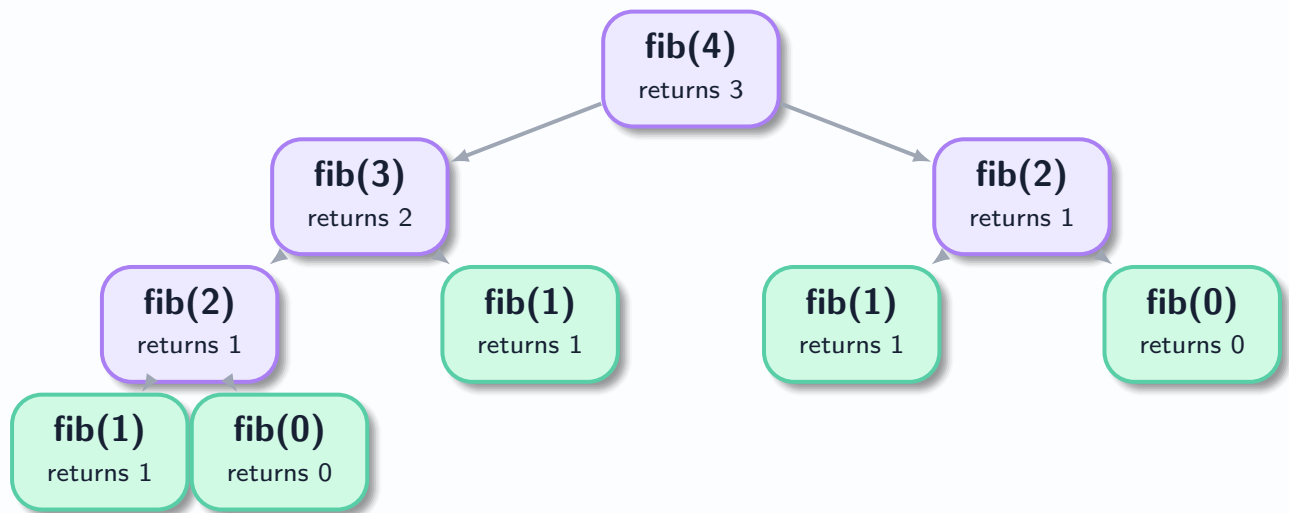
```
1 def fib(n: int) -> int:
2     """Calculate the nth Fibonacci number using recursion."""
3     if n == 0 or n == 1:
4         return n
5     return fib(n - 1) + fib(n - 2)
6
7 print(fib(4))
8 print(fib(5))
```

What Each Line Means

Line	Role in the recursion
<code>def fib(n):</code>	Creates a function. Every call has its own private value of <code>n</code> .
<code>if n == 0 or n == 1:</code>	The base cases . These stop the recursion and immediately return 0 or 1.
<code>return fib(n - 1) + fib(n - 2)</code>	The split . Python computes two smaller Fibonacci values and adds them.
<code>print(fib(4)),</code> <code>print(fib(5))</code>	Calls the function, waits for the full tree to finish, then prints the final number.

Mental Model: A Branching Tree

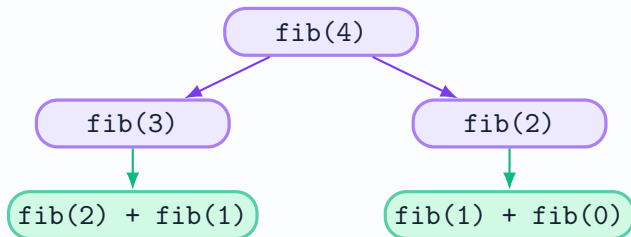
For `fib(4)`, the root call asks for two children: `fib(3)` and `fib(2)`. Any child with `n > 1` keeps splitting. Leaves are base cases, because `fib(0)` and `fib(1)` return immediately.



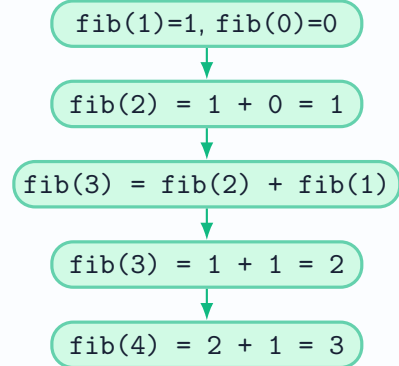
How to read the tree: start at the top, expand downward until every branch reaches `fib(0)` or `fib(1)`, then return values upward and add siblings together.

Full Calculation for `fib(4)`

1. Expand calls



2. Return and add

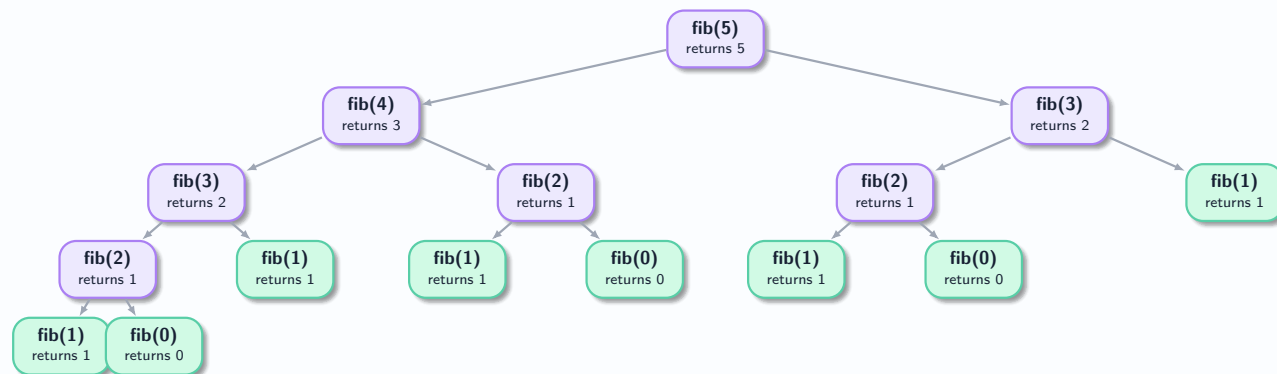


Trace Table for `fib(4)`

Step	Call	What it needs	Returns
1	<code>fib(4)</code>	<code>fib(3) + fib(2)</code>	waits
2	<code>fib(3)</code>	<code>fib(2) + fib(1)</code>	waits
3	<code>fib(2)</code>	<code>fib(1) + fib(0)</code>	waits
4	<code>fib(1)</code>	base case	1
5	<code>fib(0)</code>	base case	0
6	Left <code>fib(2)</code>	<code>1 + 0</code>	1
7	<code>fib(1)</code> inside <code>fib(3)</code>	base case	1
8	<code>fib(3)</code>	<code>1 + 1</code>	2
9	Right <code>fib(2)</code>	<code>fib(1) + fib(0)</code>	1
10	<code>fib(4)</code>	<code>fib(3) + fib(2) = 2 + 1</code>	3

Recursive Tree for fib(5)

fib(5) is one level bigger. It splits into fib(4) and fib(3). Since fib(4)=3 and fib(3)=2, the answer is 5.



Why Calls Repeat

A plain recursive Fibonacci function does not remember previous answers. So the same values are recomputed many times. In the fib(5) tree, fib(3), fib(2), fib(1), and fib(0) appear more than once.

Call type	How often in fib(5)	Meaning
fib(5)	1 time	The root problem.
fib(4)	1 time	The left child of fib(5).
fib(3)	2 times	Recomputed because both fib(5) and fib(4) need it.
fib(2)	3 times	Recomputed several times.
fib(1)	5 times	Base case returning 1.
fib(0)	3 times	Base case returning 0.

The Most Important Rule

Recursive tree rule: a node is a function call. Its children are the recursive calls it must solve. Its returned value is the sum of its children's returned values. Base-case leaves do not create children.

Final Output

3

5

So the program prints:

```
3
```

```
5
```